Ders 5 Genişletilmiş Notu: Özyineleme, Özyinelemeli İkili Arama ve Bağlı Listelere Giriş

Ders 5 Genişletilmiş Notu: Özyineleme, Özyinelemeli İkili Arama ve Bağlı Listelere Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Özyineleme Neden Gerekli?
- 2. Faktöriyel: İteratif ve Özyinelemeli Yazım
- 3. Dizi Elemanlarının Toplamı
- 4. Özyinelemeli Fonksiyon Yazmanın Adımları
- 5. Doğrudan ve Dolaylı Özyineleme
- 6. Özyinelemenin Maliyeti
- 7. Kötü Özyineleme Örneği: Fibonacci
- 8. Özyinelemeli İkili Arama
- 9. Hanoi Kuleleri Sorusu
- 10. Özyineleme Üzerine Tavsiyeler
- 11. Bağlı Listelere Giriş
- Örnek Sorular
- Dersin Toparlaması

1. Özyineleme Neden Gerekli?

Hoca derse, özyinelemeli (recursive) fonksiyonların veri yapıları ve algoritmalarda farklı durumlarda ihtiyaç duyulan bir araç olduğunu söyleyerek başladı. Sıralama algoritmalarından quicksort ve merge sort, iteratif olarak yazılması çok güç olan ama özyinelemeli yazıldığında kolaylaşan algoritmalardır. Ağaçlarla ve graflarla ilgili birçok algoritma da aynı durumdadır: iteratif olarak tasarlamak zor, özyinelemeli olarak yazınca daha kolay ve anlaşılırdır.

Bu yüzden dersin amacı, ileriki konularda ihtiyaç duyulduğunda kullanabilmek için programlama derslerinde görülen özyineleme mantığının üzerinden yeniden geçmektir.

Hocanın baştan koyduğu ilke şudur: Bir problem iteratif yöntemlerle, yani bir döngüyle çözülebiliyorsa döngüyle çözülmelidir. Özyineleme, bazı algoritmalarda gerçekten ihtiyaç duyulduğu için kullanılır.

2. Faktöriyel: İteratif ve Özyinelemeli Yazım

2.1 Faktöriyelin Tanımı

Örnek problem, dışarıdan alınan bir x sayısının faktöriyelini hesaplayıp ekrana yazan bir programdır. Ana program sayıyı alır, faktöriyel fonksiyonunu çağırır ve sonucu yazdırır.

Faktöriyelin tanımı şöyledir:

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & n > 0 \end{cases}$$

Bu tanım geriye doğru açılır. $4!$'i hesaplamak için $3!$'e ihtiyaç vardır:

$$4! = 4 \cdot 3! = 4 \cdot 3 \cdot 2! = 4 \cdot 3 \cdot 2 \cdot 1!$$

$3!$'i hesaplamak için $2!$'e, $2!$'i hesaplamak için $1!$'e ihtiyaç vardır. Genel olarak $n!$'in hesabında $n \cdot (n-1)!$ gerekir. Bu yüzden $n!$ için çağrılan fonksiyon $(n-1)!$ için kendini çağırabilir, o da $(n-2)!$ için çağırır ve böyle geriye doğru gidilir. Bu yapı özyinelemeye uygundur.

2.2 İteratif Faktöriyel

Hoca önce klasik, döngülü hâli yazdı. Sayının pozitif verildiği varsayıldı; negatif sayı durumu bu aşamada ele alınmadı.

```
int factorial(int n)
{
    int i, result = 1;
    for (i = 1; i <= n; i++)
        result = result * i;
    return result;
}
```

`i` 1'den n'ye kadar değişirken sonuç `i` ile çarpıla çarpıla gider. $4!$ için:

`i`	İşlem	`result`
başlangıç		1
1	$1 \cdot 1$	1
2	$1 \cdot 2$	2
3	$2 \cdot 3$	6
4	$6 \cdot 4$	24

Döngüden çıkıldığında $4! = 24$ elde edilir.

> **Ders sonrası not:** Hoca döngü koşulunu sözlü olarak "i n'den küçük olduğu sürece" diye anlattı, ama yaptığı iz `i = 4` adımını da içeriyor. Doğru koşul `i <= n`'dir; `i < n` ile fonksiyon $4!$ yerine $3! = 6$ döndürür. Kod `gcc -Wall` ile derlenip çalıştırıldı: `factorial(4)` 24 verir.

2.3 Özyinelemenin Özü

Özyinelemede yapılan iş, aynı işi yapan fonksiyonu daha küçük problemler için tekrar tekrar çağırmaktır. Faktöriyelde $n!$ için $(n-1)!$, $(n-1)!$ için $(n-2)!$ çağrılır. Her seferinde aynı faktöriyel fonksiyonu, bir adım geriden çağrılmış olur.

Önce durma koşulu olmadan düşünelim. `factorial(3)` çağrıldığında fonksiyon "3 çarpı `factorial(2)`" der. O "2 çarpı `factorial(1)`", o "1 çarpı `factorial(0)`", o da "0 çarpı `factorial(-1)`" der. Bu sonsuza kadar sürer.

Döngüde bunu sınır koyarak engelliyorduk: `i`'yi 1'den başlatıp n'ye kadar götürüyorduk. Özyinelemeli fonksiyonda da aynı işi görecek bir **durma noktası** olmalıdır, yoksa fonksiyon sonsuza kadar kendini çağırır. Durma noktası, tanımdaki bağıntıdan okunur: $n = 0$ ise $n! = 1$'dir.

2.4 Özyinelemeli Faktöriyel

```
int factorial(int n)
{
    if (n < 0)
        return -1;                  /* negatif değer: hata */
    if (n == 0)
        return 1;                   /* temel durum */
    else
```

```
        return n * factorial(n - 1);   /* özyinelemeli durum */
}
```

Negatif değerler için faktöriyel hesaplanamayacağı için fonksiyon -1 döndürür; çağıran taraf bu değeri görünce hata mesajı verebilir. $n = 0$ olduğunda 1 döndürülür ve özyineleme durur.

`factorial(3)` için çağrı ve dönüş sırası:

```
factorial(3) -> 3 * factorial(2)
  factorial(2) -> 2 * factorial(1)
    factorial(1) -> 1 * factorial(0)
      factorial(0) -> 1                (temel durum)
    factorial(1) = 1 * 1 = 1
  factorial(2) = 2 * 1 = 2
factorial(3) = 3 * 2 = 6   -> ana programa döner
```

Önce `factorial(2)`, sonra `factorial(1)`, sonra `factorial(0)` çağrılır. Dönüşte her fonksiyon bir üsttekine döner: `factorial(0)` biter ve `factorial(1)`'e döner, o biter ve `factorial(2)`'ye döner, o da `factorial(3)`'e döner. `factorial(3)` ise kendisini çağıran ana programa döner. Sonuç 6'dır.

Her iki yazım da aynı işi yapar. Döngüde tekrarlanan iş, özyinelemeli hâlde fonksiyonun daha küçük değerler için (n için $n - 1$) kendini çağırmasıyla tekrarlanır.

Bu fonksiyonda `n * factorial(n - 1)` içindeki `factorial(n - 1)` çağrısı **özyinelemeli kısımdır** (recursive part). `n == 0` kontrolü ise **temel durumdur** (base case): fonksiyonun sonsuza kadar dönmesini engelleyen yerdir.

3. Dizi Elemanlarının Toplamı

3.1 Özyinelemeli ve İteratif Yazım

İkinci örnek, bir dizinin elemanlarının toplamıdır. Dizi `a = {3, 5, 7, 2}` olsun; istenen $3 + 5 + 7 + 2$'dir. Bu da hem döngüyle hem özyinelemeyle yazılabilen bir problemdir. Hoca ana programı yazmadan yalnız fonksiyonu yazdı.

```
int sumOfArray(int a[], int i)
{
    if (i < 0)
        return 0;                              /* temel durum */
    else
        return a[i] + sumOfArray(a, i - 1);    /* özyinelemeli durum */
}
```

`i` 0'dan küçükse fonksiyondan çıkılır. Değilse `a[i]`, `i - 1`'inci elemana kadar olan elemanların toplamıyla toplanarak sonuç elde edilir.

İteratif hâl:

```
int sumOfArray(int a[], int n)
{
    int i, sum = 0;
    for (i = 0; i <= n; i++)
        sum += a[i];
    return sum;
}
```

Hoca iteratif hâli önce `i < n` ile yazdı. Sonra iki fonksiyonun aynı işi yapması için ana programda ikisini de `n - 1` ile çağırdı ve döngü koşulunu `i <= n` yaptı. Özyinelemeli fonksiyon `n - 1`'den başlayıp geriye gittiği için bu şekilde iki fonksiyon aynı sonucu verir.

> **Ders sonrası not:** İteratif fonksiyonun ikinci parametresi burada son indistir, eleman sayısı değildir. Fonksiyon eleman sayısıyla çağrılacaksa döngü `i < n` olmalıdır. Her iki fonksiyon `gcc -Wall` ile derlenip `n - 1 = 3` ile çalıştırıldı; ikisi de 17 verir.

3.2 Özyinelemeli Toplamın İzlenmesi

Hoca, fonksiyon adını her seferinde uzun uzun yazmamak için `sumOfArray`'i kısaca `S` diye yazdı. Fonksiyon $i = 3$ ile çağrılır.

- `S(a, 3)`: $i = 3$, 0'dan küçük değil. `a[3] + S(a, 2)` hesaplanmalıdır.
- `S(a, 2)`: `a[2] = 7`, `7 + S(a, 1)`.
- `S(a, 1)`: `a[1] = 5`, `5 + S(a, 0)`.
- `S(a, 0)`: `a[0] = 3`, `3 + S(a, -1)`.
- `S(a, -1)`: $i < 0$, temel durum, 0 döner.

Buradan sonra toplanarak geri dönülür:

Çağrı	İfade	Dönen değer
`S(a, -1)`	temel durum	0
`S(a, 0)`	$3 + 0$	3
`S(a, 1)`	$5 + 3$	8
`S(a, 2)`	$7 + 8$	15
`S(a, 3)`	$2 + 15$	17

Ana programa dönen değer 17'dir; dizinin elemanlarının toplamı 17'dir. Hoca tahtada önce ilk adımı yanlışlıkla "a'nın ikincisi" diye yazdığını ve fonksiyon adını da yanlış yazdığını fark edip düzeltti, öğrencilere de "hiç uyarmıyorsunuz" diyerek bu tür hataları söylemelerini istedi.

4. Özyinelemeli Fonksiyon Yazmanın Adımları

Hoca iki örnekten sonra özyinelemeli algoritma yazmanın adımlarını sıraladı:

1. **Temel durumu belirle (base case).** Temel durum, özyinelemenin durma noktasıdır. Dizi toplamında temel durum `if (i < 0) return 0;` satırıdır: artık geriye doğru çağırmaya devam etme, burada dur. İşlem yapılmayacak noktayı belirleyen bu koşula base case denir.
2. **Özyinelemeli durumu belirle (recursive case).** Dizi toplamında bu, `sumOfArray` fonksiyonunun yeniden çağrıldığı `a[i] + sumOfArray(a, i - 1)` kısmıdır.
3. **Özyinelemenin sonlandığından emin ol (ensure recursion terminates).** Temel durumu yazmak yetmez. Çağrılar hiçbir zaman temel duruma ulaşmıyorsa, örneğin indis hiçbir zaman 0'ın altına inmiyorsa, program sonsuza kadar döner. Bu durumda algoritmanın üzerinden yeniden geçip bir çözüm üretmek gerekir. Özyinelemeli işlemlerin sonunda gerçekten temel duruma varılıp varılmadığı kontrol edilmelidir.

Bu adımlara dikkat edildiğinde problem çözülmüş olur.

5. Doğrudan ve Dolaylı Özyineleme

5.1 Doğrudan Özyineleme

Fonksiyon kendi kendini doğrudan çağırıyorsa buna **doğrudan özyineleme (direct recursion)** denir. Faktöriyel buna örnektir. Hocanın genel örneği: Özyinelemeli bir `F` fonksiyonu bir z değeri için çağrılmış olsun, o da örneğin $z/2$ için yine `F`'yi çağırsın. Bu doğrudan çağırmadır.

5.2 Dolaylı Özyineleme

İkinci tür **dolaylı özyinelemedir (indirect recursion)**. Çok az kullanılır ve çok dikkatli takip edilmesi gerekir. Hocanın örneği:

```
int f2(int x);

int f1(int x)
{
    if (x == 1)
        return x;
```

```
    else
        return f2(x);
}

int f2(int x)
{
    return f1(x);
}
```

`f1`, `f2`'yi çağırır; `f2` de yeniden `f1`'i çağırır. Fonksiyon kendini doğrudan çağırmadığı hâlde özyineleme dolaylı yoldan oluşmuş olur. Hoca bunun iyi bir şey olmadığını, böyle yazıldığında nasıl ilerlediğinin çok iyi takip edilmesi gerektiğini söyledi.

> **Ders sonrası not:** Bu örnek, sonlanmanın neden takip edilmesi gerektiğini de gösterir. `x` değeri hiçbir çağrıda değişmediği için `f1(1)` dışındaki her çağrı (örneğin `f1(2)`) `f1` ile `f2` arasında sonsuza kadar gidip gelir ve sonunda stack taşar. Gerçek bir dolaylı özyinelemede çağrılardan en az birinin argümanı temel duruma doğru değişmelidir. Koda eklenen `int f2(int x);` bildirimi, C'de `f1` içinde `f2`'nin tanımından önce çağrılabilmesi için gereklidir.

6. Özyinelemenin Maliyeti

6.1 Ne Zaman Özyineleme Kullanılmalı?

Özyineleme, aynı işin tekrar ettiği durumlarda kullanılır. Faktöriyelde aynı hesap daha küçük bir parça için tekrarlanır, bu yüzden özyinelemeli yazmak doğru görünür. Ama artıları ve eksileri düşünülmelidir. Faktöriyeli özyinelemeli yazmanın aslında bir anlamı yoktur: iteratif hâli de aynı temizlikte ve basitlikte yazılır. Faktöriyelin özyinelemeli hâli, yalnız özyinelemeye kolay bir örnek olduğu için gösterilir.

Buna karşılık ileride görülecek quicksort algoritması özyinelemeli yazılmazsa altından kalkmak çok zordur, çünkü algoritmik olarak özyinelemeli çözüm çok daha basit ve rahat anlaşılır. Özyineleme mecbur kalındığında kullanılmalıdır.

6.2 Stack Kullanımı

Her özyinelemeli çağrı, fonksiyonun yeni bir kopyasını belleğin **stack** bölgesinde oluşturur. Değişkenler ve argümanlar bu bölgede tutulur. `factorial(3)` için stack'te yer ayrılır, çünkü ana program bu fonksiyonu çağırmıştır. Sonra `factorial(2)` için yeni bir yer açılır ama `factorial(3)`'ün yeri hâlâ durmaktadır. Sonra `factorial(1)` için ayrı bir yer tutulur. Yani 3 için kullanılan yer serbest bırakılmadan 2 için, 2 için kullanılan yer bırakılmadan 1 için yeni yer gerekir ve bu aşağı doğru devam eder.

Programlarda bazen görülen **stack overflow** hatası buradan gelir. Hocanın saydığı olası sebepler:

- Özyinelemeli fonksiyon özyinelemeden çıkamıyordur, yani temel duruma ulaşamıyordur.
- Fonksiyon kendi kendini çok fazla sayıda çağırmıştır.
- Hoca ayrıca, özyineleme olmadan da bir fonksiyonun sonsuz döngüde kalmasıyla bu hatanın görülebileceğini söyledi.

> **Ders sonrası not:** Hoca stack overflow için "özyineleme kullanmanıza gerek yok, fonksiyon sonsuz döngüde kalıyordur" dedi. İçinde fonksiyon çağrısı olmayan sıradan bir sonsuz döngü stack'i büyütmez; program yalnız takılı kalır. Stack overflow, sonlanmayan ya da çok derin özyinelemeden veya stack'te çok büyük yerel değişkenler tanımlanmasından kaynaklanır.

6.3 Zaman Maliyeti: Overhead

İkinci sorun, stack'in yönetilmesi gerektiğidir. Her fonksiyon çağrısında parametreler aktarılır, nerede kalındığı (dönüş adresi) saklanır, yerel değişkenler oluşturulur. Özyinelemeli her çağrıda bunların tekrar tekrar yapılması bir yavaşlık, yani ek yük (overhead) getirir. Problem

iteratif olarak çok kolay çözülebiliyorsa bu gereksiz bir yavaşlıktır. Aynı işi yapan iki algoritmadan özyinelemeli olan genelde daha yavaştır.

Hoca tahtaya şunu özet olarak yazdı: Her özyinelemeli çağrıda fonksiyonun yeni bir kopyası stack bölgesinde tutulur; değişkenler ve argümanlar stack'te saklanır. Bu hem yer tutar hem de işin yapılması zaman alır. Aynı problemi çözen iki yaklaşım varsa, özyinelemeli çağırma stack'te gereksiz yere fazla yer tuttuğu ve stack işlemleriyle vakit harcandığı için daha yavaştır.

Bazı durumlarda ise problemi düz yazmak çok mümkün olmaz, çünkü stack gibi çalışan bir çözüme ihtiyaç vardır: bir değeri hesaplamak için bir önceki durumu, ondan önceki durumu bilmek gerekir. Böyle problemlerde özyineleme mutlaka kullanılır.

> **Ek bilgi (derste anlatılmadı):** Fonksiyon çağrıları için tutulan bu bölgeye çağrı yığını (call stack), her çağrı için ayrılan parçaya da yığın çerçevesi (stack frame) denir. Son çağrılan fonksiyon ilk biter; bu son giren ilk çıkar (LIFO) davranışıdır. Bu yüzden özyinelemeli bir çözüm, açık bir stack veri yapısı kullanan bir döngüyle de yazılabilir.

6.4 Sınavlarda Gereksiz Özyineleme

Aradan sonra hoca şu uyarıyla devam etti:

> **Hocanın vurgusu:** Sınavlarda her şeyi özyinelemeli yazmaya çalışan öğrencilerle çok karşılaşılıyor. Bunu yapmayın. Önce bu problemi gerçekten özyinelemeli yazmanız gerekip gerekmediğine karar verin.

7. Kötü Özyineleme Örneği: Fibonacci

7.1 Birbirine Bağlı Alt Problemler

Her çağrıda stack'te yer tutma, değişkenleri, parametreleri ve dönüş adreslerini saklama maliyetinin yanında bir sorun daha vardır. Hoca buna özellikle dikkat çekti: Özyinelemeli fonksiyon yazarken alt problemler birbiriyle ilişkiliyse, aynı işi yapan fonksiyonlar defalarca çağrılabilir. Bu sık karşılaşılan bir sıkıntıdır ve çok güzel örneği Fibonacci problemidir.

Fibonacci dizisinde ilk sayılar 0, 1, 1'dir ve her sayı kendinden önceki iki sayının toplamıdır: $1 + 1 = 2$, $1 + 2 = 3$, $2 + 3 = 5$ gibi. Yineleme bağıntısı (recurrence relation) üç durumludur:

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

Bu bağıntı bakar bakmaz özyinelemeli yazılabilecek gibi görünür:

```
int fib(int n)
{
    if (n == 0)
        return 0;                       /* durma koşulu */
    else if (n == 1)
        return 1;                       /* durma koşulu */
    else
        return fib(n - 1) + fib(n - 2);
}
```

Hoca yer olmadığı için tahtada `fib(n - 1)` ile `fib(n - 2)`'yi alt alta yazdı; normalde yan yana yazılabilir.

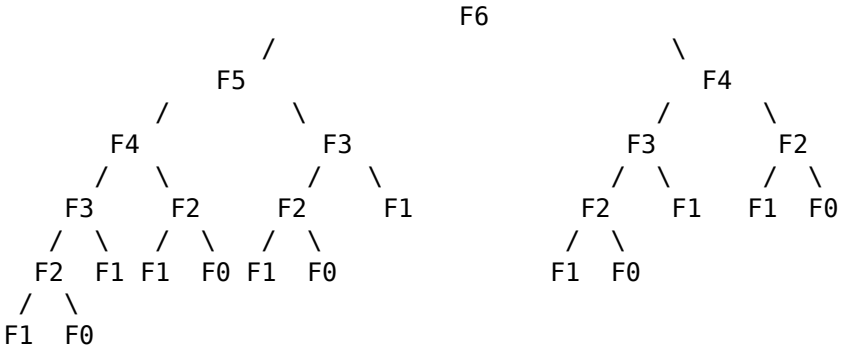
7.2 Son `else` Gerekli mi?

Hoca burada öğrencilerin sormasını beklediği bir soruyu kendisi sordu: `n == 0` ise `return 0` ile zaten fonksiyondan çıkılıyor, `n == 1` ise `return 1` ile çıkılıyor. O hâlde son satırın başına neden `else` yazılıyor?

Soru: Bir öğrenci, 2 girildiğinde 1 ve 0 elde edildiği için olabileceğini, `else` olmadan Fibonacci'nin ikinci sayısının hesaplanamayabileceğini söyledi. **Cevap:** Hoca, `else` olmadan da programın doğru çalışacağını söyledi. İki koşuldan biri sağlanıyorsa fonksiyon `return` ile çıkıp gitmiştir; ikisi de sağlanmıyorsa akış zaten son satıra düşer. `else` tamamen okunabilirlik için yazılır: kodu okuyan kişi hesap yapmadan "0 değilse, 1 değilse buraya girer" sonucunu bakar bakmaz görür.

7.3 `fib(6)` Çağrı Ağacı

Hoca adı kısaltıp `F` yazdı ve `F(6)`'nın hesabını izledi. `F(6)`, `F(5) + F(4)`'ü çağırır. `F(5)`, `F(4) + F(3)`'ü; `F(4)`, `F(3) + F(2)`'yi çağırır ve böyle devam eder. `F(1)` ve `F(0)` doğrudan 1 ve 0 döndürür.



Hocayı rahatsız eden, aynı değerlerin defalarca hesaplanmasıdır. Hoca çağrıları ağaç üzerinde saydı: $F(4)$ iki kez, $F(3)$ üç kez çağrılır. $F(1)$'i saydığında önce 6, sonra gözden kaçanlarla 8 buldu: yalnız $F(1)$ için 8 kez aynı hesap yapılır. Bu, özyinelemenin yanlış, kötü kullanımına bir örnektir.

Çağrı sayıları (C programına sayaç eklenerek ve python ile doğrulandı):

Değer	$F(6)$	$F(5)$	$F(4)$	$F(3)$	$F(2)$	$F(1)$	$F(0)$
Çağrı sayısı	1	1	2	3	5	8	5

Toplam 25 çağrı ve 12 toplama işlemi yapılır; sonuç $F(6) = 8$'dir.

> **Ders sonrası not:** Hoca $F(2)$ için önce 4, sonra 5 ve 6 kez; özet kısmında da "3 için 2 defa" dedi. Doğru sayılar $F(3)$ için 3, $F(2)$ için 5'tir. $F(4)$ için 2 ve $F(1)$ için 8 doğrudur.

Hoca, bu hesabın yapıldığını ve Fibonacci'de yapılan toplama sayısının yaklaşık 1.6^n ile büyüdüğünü söyledi.

> **Ders sonrası not:** Hoca bunu "Fibonacci 6 için 1.6 üzeri n tane toplama" diye söyledi; bu ifade tek bir n için tam sayı değil, büyüme hızıdır. $F(n)$ için yapılan toplama sayısı $F(n+1)-1$'dir ($n = 6$ için 12, $1.6^6 \approx 16.8$ değil). Ardışık toplama sayılarının oranı python ile hesaplandı: 1.714, 1.667, 1.65, ... diye azalarak $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$'e yaklaşır. Yani "yaklaşık 1.6^n" bir büyüme hızı olarak doğrudur.

Kod çok şık görünür: $F(6)$, kendinden önceki iki değer olan $F(5)$ ile $F(4)$'ün toplamıdır ve bu koddan bakar bakmaz anlaşılır. Ama alt problemler birbirine bağlı olduğu için aynı şeyler gereksiz yere defalarca çağrılır. $F(2)$'yi bir kez hesaplayıp kullanmak varken onu defalarca hesaplamak gerekir. Bu problem döngü içinde, önceki değerleri bilerek bir sonrakini hesaplayarak şık biçimde çözülebildiği için özyinelemeli kullanım uygun değildir.

7.4 Çıkan Dersler

Hocanın özyinelemeli fonksiyon yazarken dikkat edilmesini istediği iki nokta:

1. Aynı iş özyinelemesiz bir algoritmayla açık ve basit biçimde çözülebiliyorsa, özellikle stack kullanılarak çözülmesine ihtiyaç duyulmayan problemlerde, özyinelemesiz yazılmalıdır. Böylece fonksiyona gidip gelme ek yükünden kaçınılır.
2. Fibonacci'de görüldüğü gibi aynı değerler için fonksiyonun defalarca çağrılıp çağrılmadığına bakılmalıdır, çünkü bu gereksiz işlem yapılmasına ve zaman kaybına yol açar.

8. Özyinelemeli İkili Arama

8.1 İkili Aramanın Mantığı

Hoca, özyinelemesiz çözülse de yerinde olan ve çok sık özyinelemeli çözümüyle karşılaşılan bir algoritma olarak ikili aramayı (binary search) ele aldı.

İkili aramanın şartı, dizinin sıralı olmasıdır; küçükten büyüğe ya da büyükten küçüğe sıralı olması fark etmez. Normalde bir eleman aranırken bütün elemanlara tek tek bakılır. İkili aramada amaç gereksiz kontrolleri engellemektir: Bakılan aralığın tam ortasındaki elemana bakılır. Aranan sayı ortadakinden büyükse, ortadakinden küçük olan bütün elemanlardan da büyüktür; onlara hiç bakmaya gerek yoktur. Böylece her adımda bir grup eleman hiç bakılmadan elenir. Dizi her adımda ikiye bölünerek elendiği için algoritmaya ikili arama denir.

8.2 Örnek: Dizide Olan Bir Eleman

Dizi: `{3, 5, 7, 11, 13, 17, 21, 29}`, $n = 8$, aranan $x = 29$. Üç değişken tutulur: `left`, `right`, `mid`. Başta `left = 0`, `right = 7`'dir.

`left`	`right`	`mid`	`a[mid]`	Karar
0	7	$7/2 = 3$	11	$29 > 11$: 3, 5, 7'ye bakmaya gerek yok, `left = mid + 1 = 4`
4	7	$11/2 = 5$	17	$29 > 17$: 13'e de bakmaya gerek yok, `left = 6`
6	7	$13/2 = 6$	21	$29 > 21$, `left = 7`
7	7	7	29	Bulundu

Tamsayı bölmesinde $(0 + 7)/2 = 3$, $(4 + 7)/2 = 5$, $(6 + 7)/2 = 6$ olur (ara adım eklendi).

8.3 Örnek: Dizide Olmayan Bir Eleman

Aynı dizide $x = 8$ aranır.

`left`	`right`	`mid`	`a[mid]`	Karar
0	7	3	11	$8 < 11$: 13, 17, 21, 29'u ele, `right = mid - 1 = 2`
0	2	1	5	$8 > 5$: 3'e de bakmaya gerek yok, `left = 2`
2	2	2	7	$8 > 7$, `left = 3`
3	2			`left > right`: eleman dizide yok

Aranan sayı ortadakinden küçükse, aralığı daraltmak için bakılan aralığın sonu `mid - 1` yapılır. Büyükse başlangıcı `mid + 1` yapılır.

`left = 3`, `right = 2` olduğunda bunun bir anlamı yoktur: Bakılan aralığın başlangıç indisi, bitiş indisinden büyük olamaz. Bitiş her zaman başlangıçtan büyük ya da ona eşit olmalıdır. `left > right` olduğunda algoritma durdurulur. Yani ya eleman bulununca çıkılır ya da `left > right` olunca elemanın dizide olmadığı anlaşılır ve çıkılır.

Soru: Hoca bu algoritmanın daha önce gösterilip gösterilmediğini sordu. **Cevap:** Öğrenciler, bilgisayar bilimlerine giriş dersinde gördüklerini söyledi.

8.4 Özyinelemeli Kod

Bu algoritma programlama derslerinde genelde `while` döngüsüyle adım adım gösterilir. Anlaşılır olduğu için özyinelemeli hâli de sıkça gösterilir. Hoca kendi görüşünün, bunu da döngüyle yapmanın daha verimli olduğu yönünde olduğunu söyledi; ama çok kullanıldığı için özyinelemeli hâline bakıldı.

Başlangıçta `low = 0` dizinin başını, `high = n - 1` son elemanın indisini gösterir. Her adımda bu değerler değişir. `mid`, o an bakılan bölgenin tam ortasıdır: `(low + high) / 2`.

Bu algoritmada birden fazla durma koşulu vardır:

- **Temel durum 1:** `low > high` ise eleman bulunamamıştır, -1 döndürülür.
- **Temel durum 2:** `a[mid] == x` ise eleman bulunmuştur, elemanın indisi olan `mid` döndürülür.

Diğer iki durumda fonksiyon kendini çağırır:

- `a[mid] > x` ise sağ taraf elenir. Aralığın başlangıcı (`low`) aynı kalır, sonu `mid - 1` olur.
- Aksi hâlde, yani `a[mid] < x` ise sol taraf elenir. Aralık `mid + 1`'den başlar, `high`'a kadar gider.

```c
int binarySearch(int a[], int low, int high, int x)
{
    int mid;
    if (low > high)
        return -1;                                  /* temel durum 1: yok */
    mid = (low + high) / 2;
    if (x == a[mid])
        return mid;                                 /* temel durum 2: bulundu */
    else if (a[mid] > x)
        return binarySearch(a, low, mid - 1, x);    /* sol yarı */
    else
        return binarySearch(a, mid + 1, high, x);   /* sağ yarı */
}

/* İlk çağrı: binarySearch(a, 0, n - 1, x) */
```

> **Ders sonrası not:** Özyinelemeli çağrıların başında `return` olmalıdır. `return` yazılmazsa alt çağrının bulduğu indis çağırana iletilmez ve C'de fonksiyon değer döndürmeden biter (tanımsız davranış; `gcc -Wall` bunu "control reaches end of non-void function" uyarısıyla gösterir). Kod `return` ile derlenip çalıştırıldı: 29 için 7, 8 için -1, aşağıdaki örnekte 5 için 1 döndü.
>
> **Ek bilgi (derste anlatılmadı):** Her adımda aralık yarıya indiği için n elemanlı dizide en fazla yaklaşık $\log_2 n + 1$ karşılaştırma yapılır; sıralı aramada bu sayı n'dir. Çok büyük dizilerde `low + high` toplamı `int` sınırını aşabileceği için `mid` çoğu zaman `low + (high - low) / 2` diye yazılır.

8.5 Kodun Adım Adım İzlenmesi

Hoca kodu beş elemanlı bir dizi üzerinde izledi: `a[0] = 3`, `a[1] = 5`, `a[2] = 7`. Önce $x = 3$ ile başladı, sonra bir adım daha görülsün diye $x = 5$'e geçti.

Başta `low = 0`, `high = 4`.

low	high	mid	a[mid]	Karar
0	4	2	7	$5 \neq 7$; $7 > 5$: `binarySearch(a, 0, 1, 5)`
0	1	$1/2 = 0$	3	$3 < 5$: `binarySearch(a, 1, 1, 5)`
1	1	1	5	$5 = 5$: `return 1`

Her çağrıda önce `low > high` kontrolü yapılır; bu örnekte hiçbir adımda sağlanmaz. Son çağrıda `a[1] = 5` aranan değere eşit olduğu için `mid = 1` döndürülür ve bu değer çağrılar boyunca ana programa ulaşır.

Özetle, aynı fonksiyon bakılan aralığı değiştirmek için çağrılır: `a[mid] > x` ise dizinin sol tarafına, küçükse sağ tarafına bakılır. Eşitse eleman bulunduğu için durulur; bulunamazsa `low > high` olur ve yine durulur.

9. Hanoi Kuleleri Sorusu

Hoca bir soru verdi ve çözmedi; öğrencilerin kendi başlarına çözmesini istedi. Bu soru hocanın da ikinci sınıftayken kendisine sorulmuş bir soruymuş. Kendi başına, bir yere bakmadan çözmek kolay değildir ama çözülünce zevk veren bir sorudur.

> **Hocanın vurgusu:** Hanoi Kuleleri'nin çözümüne internetten bakmayın; adını yazar yazmaz çözümü görürsünüz. Demolarına bakabilirsiniz, ama algoritmayı kendiniz kurmaya çalışın.

Hoca soruyu küçük çocukların oynadığı renkli halka oyuncağına benzeterek anlattı: Bir çubuğa geçirilmiş, büyükten küçüğe, büyük en altta olacak biçimde dizilen plastik renkli halkalar.

Problem şöyledir:

- A çubuğunda dört halka vardır: en altta kırmızı, onun üstünde pembe, onun üstünde yeşil, en üstte mavi. Kırmızı en büyük, mavi en küçüktür.
- Amaç: Bütün halkaları A'daki sırayla B çubuğuna taşımak. İş bittiğinde B'de en altta kırmızı, üstünde pembe, onun üstünde yeşil, en üstte mavi olmalıdır.
- C çubuğu, taşıma sırasında geçici (temporary) olarak kullanılabilir.

Kurallar:

1. Her seferinde yalnız bir disk oynatılabilir. Dördünü birden alıp yerleştirmek ya da aynı anda hem maviyi hem yeşili oynatmak yoktur.
2. Küçük bir diskin üzerine asla büyük bir disk konamaz.
3. Aradan disk çekilemez, yalnız en üstteki disk hareket ettirilebilir. Örneğin pembeyi hareket ettirebilmek için önce mavinin, sonra yeşilin üstünden kalkmış olması gerekir.

Hoca başlangıçta yalnız mavinin hareket ettirilebileceğini gösterdi: mavi bir çubuğa konursa üstte yeşil kalır, yeşil de başka bir çubuğa konabilir. Çözüm derste verilmedi; aşağıdaki Örnek Sorular bölümünde kesinleştirilmiş çözüm var.

10. Özyineleme Üzerine Tavsiyeler

İkinci aradan sonra hoca özyineleme konusunu şu tavsiyelerle kapattı. Sınav kağıtlarında gereği olmadığı hâlde zorlama özyineleme kullanımları görülüyor ve öğrenciler bu durumlarda çoğunlukla hata yapıyor.

Birinci tavsiye: Özyinelemeli fonksiyon yazmayı öğrenin. Farklı örnekler deneyerek kendi kendinize özyinelemeli fonksiyon yazın. Kendini çağıran fonksiyonun düşünce mantığı, döngüyle çözülen problemlerden farklıdır; temel durumu ve özyinelemeli durumu iyi planlayıp kurgulamak gerekir. Hanoi Kuleleri bu mantığın oturmasına yardımcı olur. Başka alıştırma önerileri:

- Bir sayının n'inci kuvvetinin hesaplanması.
- Bir kelimenin palindrom olup olmadığının bulunması. Palindrom, sağdan ve soldan okunduğunda aynı olan kelimedir; "kabak" ve "yay" buna örnektir.

Graf ve ağaçlarla ilgili özyinelemeli işlemler yeri geldikçe görülecek; şimdilik döngüyle çözülen bazı problemler özyinelemeli çözülerek pratik yapılabilir. Bazı yerlerde özyineleme mecburidir, çünkü problem başka türlü çözülmez.

İkinci tavsiye: Verimli algoritma yazın.

> **Hocanın vurgusu:** Veri yapıları ve algoritmalar dersiyle bir adım öteye geçtiniz. Artık önemli olan algoritmanın çalışması değil; zamanı ve yeri iyi kullanan, verimli, rahat anlaşılır bir algoritma yazmanız bekleniyor. Bunun bir parçası da gereksiz işlem yapmamaktır. Algoritmanıza karmaşık, sofistike bir görüntü vermek için olur olmaz her yerde özyineleme kullanmayın; yalnız gerçekten gerektiğinden emin olduğunuz yerlerde kullanın.

11. Bağlı Listelere Giriş

11.1 Dersin İki Ayağı ve Dizi

Hoca ilk derste söylenenleri hatırlattı: Bu ders bir bütündür. Hem çok sık kullanılan temel algoritmalar, hem de daha önemlisi verimli tasarım yapmayı kolaylaştıran veri yapıları görülür.

Birinci sınıfta görülen ilk veri yapısı dizidir (array). Bir sınıftaki üç öğrencinin not ortalaması istenirse üç değişken tanımlanır (A, B, C), toplanır, üçe bölünür. Ama bir sınıftaki öğrencilerin ortalaması istendiğinde sınıfta 20 de olabilir, 50 de. Bu kadar değişken tanımlamak ve kullanmak işi karıştırır. Dizi, aynı amaçla kullanılan birden fazla veriyi bir arada tutar: Örneğin en fazla 50 öğrenci olabilecekse 50 elemanlık bir dizi tanımlanır.

11.2 Statik Diziden Dinamik Bellek Ayırmaya

Statik dizinin sorunu, kaç öğrenci olacağının belli olmamasıdır. Yer en fazla öğrenci sayısına göre açılır: 5 öğrenci de olsa, 50 de olsa 500'lük yer tutulur. Bu, program çalışmadan önce ne kadar yer tutulacağı belli olan **statik bellek ayırmadır (static allocation)**.

Bunu iyileştirmek için **dinamik bellek ayırma (dynamic memory allocation)** yapılır. Dışarıdan verilen n sayısına göre, o anki ihtiyaç kadar yer ayrılır: 100 öğrenci varsa 100'lük, 50 öğrenci varsa 50'lik. Sistem bunun için belirli bir başlangıç adresinden itibaren (örneğin 1000 numaralı adresten) ardışık, n elemanlık bir blok ayırır.

Program boyunca hep n eleman için iş yapılacağı biliniyorsa en iyi çözüm budur.

11.3 Daha Dinamik Bir Yapıya İhtiyaç

Bazı uygulamalarda farklı bir mantık gerekir. Hoca iki örnek verdi:

- Kaç eleman olacağı baştan bilinmez. Eleman geldikçe eklenir, işi bitince silinir.
- Graflarda kullanılan bir tür matris yapısında bir satırda iki, bir satırda üç, bir satırda bir eleman olabilir ve gerekirse ekleme ve silme yapılır.

Bir seferde n eleman için iş yapılmasının gerekmediği bu durumlarda kullanılan yapı, programlama derslerinde görülen **bağlı listedir (linked list)**. Bu yapı ağaç ve graf konularında kullanılacak.

11.4 Düğüm Yapısı

Listenin her elemanına **düğüm (node)** denir. Her düğümün bir değeri ve bir sonraki düğümü gösteren bir pointer'ı vardır.

```
struct node {
    int value;
    struct node *next;
};

struct node *p;
p = (struct node *) malloc(sizeof(struct node));
if (p == NULL) {
```

```
    /* yer ayrılamadı */
}
```

Hoca burada yapının kendisinin özyinelemeli olduğuna dikkat çekti: `struct node` tipinin içinde yine `struct node` tipini gösteren bir pointer tanımlanır. Yapı, tamamen kendi özelliklerinde bir değişkeni gösterecek biçimde tanımlanır.

`malloc` ile `sizeof(struct node)` kadar yer istendiğinde tek bir eleman için yer tutulmuş olur. Dinamik bellek ayırmada yer ayrılamazsa `NULL` döner; değilse yer tutulmuştur.

Tutulan yer bir yapıdır: bir `value` kısmı ve bir `next` kısmı vardır. `value` normal bir tamsayıdır. `next` ise aynı yapıdaki, yani yine `value` ve `next` kısımlarından oluşan başka bir değişkenin adresini tutmak için kullanılan bir pointer değişkendir.

11.5 Tek Yönlü Bağlı Liste

Hocanın örneğinde düğümler 1000, 1250 ve 1300 numaralı adreslerde durur:

```
head -> [5 | 1250] -> [7 | 1300] -> [8 | NULL]
         (1000)        (1250)        (1300)
```

1000 adresindeki düğümün `next`'ine 1250 yazılırsa bu düğüm 1250'deki düğümü gösterir. İlk düğümü de birinin göstermesi gerekir: Her zaman bir **head** vardır ve listenin ilk elemanını gösterir; listeye oradan erişilir. Yeni bir eleman için yer gerektiğinde, örneğin 1300 numaralı adres tutulur ve listede uygun bir düğüme (son elemana, ilk elemana ya da ihtiyaç olan yere) 1300 adresi gösterilirse bu düğüm de listenin bir parçası olur.

Bu yapıya **tek yönlü bağlı liste (singly linked list)** denir. En temel yapıdır ve yalnız ileriye doğru (forward only) gidilir. Her eleman bir sonrakinin adresini gösterir; son eleman kimseyi göstermez, `NULL`'ı gösterir. Head'e bakılarak ilk elemanın nerede olduğu, `NULL`'a gelindiğinde de listedeki bütün elemanlara erişildiği anlaşılır.

Her eleman için ayrı ayrı bellek ayrılır. Hoca adresleri bilerek birbirinden uzak seçtiğini söyledi: Düğümler arka arkaya da gelebilir, ama ayrıldıkları anda bellekte nerede boşluk varsa orada dururlar.

11.6 Temel İşlemler

Bağlı listede yapılabilecek temel işlemler:

1. **Ekleme (insert):** Yeni bir eleman eklemek.
2. **Silme (delete):** Artık kullanılmayan bir elemanı silmek.
3. **Arama (search):** Listede bir x elemanını aramak.
4. **Gösterme (display):** Listedeki bütün elemanları ekranda göstermek.

11.7 Avantajlar

- **Uzunluk sabit değildir.** Bu özellik dinamik bellek ayırmada da vardır. Ama orada bilgi bir blok hâlinde tutulur; bağlı listede blok olması gerekmez, düğümler bellekte "delik deşik" dağılabilir.
- **Ardışık yer gerekmez.** Bellekte yeterince büyük ardışık boş yer yoksa bile bağlı liste kullanılabilir.
- **Ekleme ve silme kolaydır.** Blok hâlinde ayrılmış bir dizide ortadan bir eleman silinirse ya yeri boş bırakılır (sık silmede dizi delik deşik olur) ya da sonraki bütün elemanlar kaydırılarak boşluk doldurulur (sıkıştırma). Bağlı listede ise örneğin 1250'deki düğüm silinecekse bu düğüm `free` ile serbest bırakılır ve 1000'deki düğüm artık 1250'yi değil 1300'ü gösterir. Yüz tane eleman olsa da hiçbiri yerinden oynatılmaz.
- Araya ekleme de aynı mantıkla yapılır. Dizide özel bir yere ekleme yine kaydırma gerektirir. Sıralı bir bağlı listede yeni bir eleman 1400 numaralı adreste olsun: 1300'ü gösteren düğüm artık 1400'ü, yeni düğüm de 1300'ü gösterir. Başa ya da sona ekleme de benzer biçimde yapılır. Bağlı listede iş birkaç pointer işlemiyle çözülür; dizide çok sayıda elemanın kaydırılması gerekebilir.

11.8 Dezavantajlar

- **Rastgele erişim (random access) yoktur.** Bu, en temel dezavantajdır. Örnekteki 8'e ulaşmak için baştan 5'e, sonra 7'ye tek tek uğrayarak gelmek gerekir. 8'in 1400'de durduğu biliniyorsa doğrudan o adrese gidilebilir; ama bağlı listede saklanan aradaki bir elemana doğrudan erişmek mümkün değildir, liste üzerinde ilerleyerek bulunur.
- **Pointer için ek yer gerekir.** Klasik dinamik bellek ayırmada tek bir pointer değişken tutulan bloğun başlangıcını gösterir ve blok üzerinde onunla ilerlenir. Bağlı listede ise her elemanın yanında bir sonrakinin adresini tutan bir pointer olmak zorundadır.
- **Önbellek (cache) avantajı yoktur.** Bilginin önbellekte olması hızlı erişim sağlar; bağlı listede bu avantaj yoktur.

Bağlı liste genellikle graf ve ağaç veri yapılarında çok kullanılır.

Soru: Bir öğrenci, bağlı listenin önbellekte kullanılamamasının temel bir sebebi olup olmadığını sordu. **Cevap:** Hoca, bağlı listenin dinamik bellek ayırmayla belleğin heap bölgesini kullandığını, öyle tasarlandığı ve planlandığı için önbellekte kullanılmadığını söyledi.

> **Ders sonrası not:** Heap bölgesindeki veri de önbelleğe alınır; bağlı listenin "önbellekte kullanılamaması" diye bir kural yoktur. Asıl sorun düğümlerin bellekte dağınık durmasıdır. Dizide ardışık elemanlar aynı önbellek satırına düşer ve birlikte önbelleğe gelir (locality). Bağlı listede bir sonraki düğüm başka bir yerde olduğu için sık önbellek ıskalaması (cache miss) olur. Bu yüzden bağlı liste üzerinde dolaşmak, aynı boyuttaki diziden genelde yavaştır.

11.9 `malloc` ve `calloc`: Pointer'ların İlk Değeri

Hoca ekleme işlemini göstermeyi planlıyordu ama süre dolduğu için bir sonraki derse bıraktı. Bu sırada sohbet penceresinden bir soru geldi.

Soru: Pointer'ların varsayılan (default) değeri `NULL` mı? **Cevap:** Hayır, pointer'ların varsayılan değeri diye bir şey yoktur. C'de iki fonksiyon vardır: `calloc` ayırdığı yeri baştan sıfırlar, `malloc` sıfırlamaz. Sıfırlanmayan yerde o bellek bölgesinde önceden ne varsa (çöp değer, garbage) o durur. Bu yüzden değişkenleri ve pointer'ları mutlaka kendiniz ilk değerle başlatmalısınız (initialize). Birisi sizin `calloc` ile yazdığınız yeri `malloc` ile açabilir; baştan sıfırlanmadığı için her şey ters gidebilir. Ayrıca bütün dillerde bellek ayırma fonksiyonlarının aynı davranacağına dair bir kural yoktur. En güvenlisi, bellek sıfırlanmıyormuş gibi davranmaktır.

> **Hocanın vurgusu:** Değişkenleri ve pointer'ları her zaman kendiniz başlatın; bellek sıfırlanmıyormuş gibi davranın.
>
> **Ders sonrası not:** Hoca "P kendisi 2500 numaralı adreste duruyorsa orada ne varsa onu gösterir" dedi. Doğrusu: İlk değer verilmemiş bir yerel pointer değişkeninin içinde çöp bir adres vardır. `malloc` ise geçerli bir adres döndürür ama o adresteki içeriği sıfırlamaz; yani yeni düğümün `next` alanı çöp değer taşır. Bu yüzden yeni düğümün `next` alanına açıkça `NULL` atanmalıdır.
>
> **Ek bilgi (derste anlatılmadı):** `calloc(adet, boyut)` iki argüman alır ve `adet * boyut` baytlık alanı sıfırlayarak ayırır; `malloc(boyut)` tek argüman alır. İkisi de yer ayıramazsa `NULL` döndürür ve ayrılan yer işi bitince `free` ile bırakılır.

Örnek Sorular

Bu derse ait hazır kare ve video olmadığı için sorular yalnız derste sözlü sorulanlardan derlendi.

Soru 1 (derste sözlü, 00:12:02)

İteratif faktöriyel fonksiyonuyla $4!$ hesaplanırken her adımda `result` ne olur?

Çözüm (derste verildi): `result = 1` ile başlanır. $i = 1$: 1; $i = 2$: $1 \cdot 2 = 2$; $i = 3$: $2 \cdot 3 = 6$; $i = 4$: $6 \cdot 4 = 24$. Döngüden çıkınca sonuç 24'tür.

Soru 2 (derste sözlü, 00:16:41)

Özyinelemeli faktöriyel fonksiyonunda durma noktası nedir? Durma noktası olmazsa `factorial(3)` ne yapar?

Çözüm (derste verildi): Durma noktası tanımdaki $0! = 1$ koşuludur: `if (n == 0) return 1;`. Bu olmazsa `factorial(3)` sırasıyla `factorial(2)`, `factorial(1)`, `factorial(0)`, `factorial(-1)`, ... diye sonsuza kadar kendini çağırır. Negatif girişe karşı ayrıca `if (n < 0) return -1;` eklenir.

Soru 3 (derste sözlü, 01:04:16)

Fibonacci fonksiyonunda `n == 0` ve `n == 1` durumları zaten `return` ile çıkıyorsa, son satırdaki `else` gerekli mi? `else` olmadan program doğru çalışır mı?

Çözüm (derste verildi): Gerekli değildir, `else` olmadan da program doğru çalışır. İki koşuldan biri sağlanıyorsa fonksiyon zaten `return` ile çıkmıştır; sağlanmıyorsa akış son satıra düşer. `else`, kodu okuyanın "0 değilse, 1 değilse buraya girer" sonucunu hesap yapmadan görmesi için, okunabilirlik amacıyla yazılır.

Soru 4 (derste sözlü, 01:08:34)

Özyinelemeli `fib(6)` çağrısında $F(4)$, $F(3)$, $F(2)$ ve $F(1)$ kaçar kez çağrılır?

Çözüm (derste verildi, sayılar düzeltildi): Hoca $F(4)$ için 2, $F(3)$ için 3, $F(1)$ için 8 buldu; $F(2)$ için 4, 5 ve 6 arasında farklı sayılar söyledi. Sayaçlı C programı ve python ile doğrulanan değerler: $F(4)$: 2, $F(3)$: 3, $F(2)$: 5, $F(1)$: 8 (ayrıca $F(0)$: 5). Toplam 25 çağrı ve 12 toplama yapılır, sonuç $F(6) = 8$'dir.

Soru 5 (derste sözlü, 01:27:30): Hanoi Kuleleri

A çubuğunda büyükten küçüğe dizili dört halka (alttan üste kırmızı, pembe, yeşil, mavi) aynı sırayla B çubuğuna taşınacak; C geçici çubuktur. Her seferinde yalnız bir disk ve yalnız en üstteki disk oynatılabilir; küçük diskin üzerine büyük disk konamaz. Nasıl yapılır?

> **Çözüm (derste verilmedi):** Hoca öğrencilerin bunu önce kendi başına denemesini istedi. Özyinelemeli fikir: n diski A'dan B'ye taşımak için önce üstteki $n-1$ diski A'dan C'ye (B geçici) taşı, sonra en büyük diski A'dan B'ye taşı, sonra $n-1$ diski C'den B'ye (A geçici) taşı. Temel durum $n = 0$'dır (taşınacak disk yok). Hamle sayısı $T(n) = 2T(n-1) + 1$, $T(0) = 0$ bağıntısından $T(n) = 2^n - 1$'dir; 4 disk için 15 hamle.

```c
#include <stdio.h>

void hanoi(int n, char from, char to, char temp)
{
    if (n == 0)
        return;                         /* temel durum */
    hanoi(n - 1, from, temp, to);       /* üstteki n-1 diski geçici çubuğa */
    printf("disk %d: %c -> %c\n", n, from, to);
    hanoi(n - 1, temp, to, from);       /* n-1 diski hedefe */
}

int main(void)
{
    hanoi(4, 'A', 'B', 'C');
    return 0;
}
```

Kod `gcc -Wall` ile derlenip çalıştırıldı. Disk 1 mavi, 2 yeşil, 3 pembe, 4 kırmızıdır. Çıktıdaki 15 hamle:

Hamle	Disk	Hareket	Hamle	Disk	Hareket
1	1 (mavi)	A → C	9	1 (mavi)	C → B
2	2 (yeşil)	A → B	10	2 (yeşil)	C → A
3	1 (mavi)	C → B	11	1 (mavi)	B → A
4	3 (pembe)	A → C	12	3 (pembe)	C → B
5	1 (mavi)	B → A	13	1 (mavi)	A → C
6	2 (yeşil)	B → C	14	2 (yeşil)	A → B
7	1 (mavi)	A → C	15	1 (mavi)	C → B
8	4 (kırmızı)	A → B			

Soru 6 (derste sözlü, 01:47:21)

Bir sayının n'inci kuvvetini özyinelemeli hesaplayan fonksiyonu yazın.

> **Çözüm (derste verilmedi):** $x^0 = 1$ temel durum, $x^n = x \cdot x^{n-1}$ özyinelemeli durumdur ($n \geq 0$ varsayıldı). `gcc -Wall` ile derlenip çalıştırıldı: `power(2, 10)` 1024, `power(3, 4)` 81 verdi.

```c
long power(long x, int n)
{
    if (n == 0)
        return 1;                    /* temel durum */
    else
        return x * power(x, n - 1);  /* özyinelemeli durum */
}
```

Soru 7 (derste sözlü, 01:47:54)

Bir kelimenin palindrom olup olmadığını (örneğin "kabak", "yay") özyinelemeli bulan fonksiyonu yazın.

> **Çözüm (derste verilmedi):** Baştaki ve sondaki karakter karşılaştırılır. Farklıysa kelime palindrom değildir. Aynıysa aradaki kısım için fonksiyon yeniden çağrılır. Bakılacak kısım bir karaktere ya da boşa inince (`left >= right`) kelime palindromdur; bu temel durumdur. `gcc -Wall` ile derlenip çalıştırıldı: "kabak" ve "yay" için 1, "veri" için 0 döndü.

```c
int isPalindrome(const char s[], int left, int right)
{
    if (left >= right)
        return 1;                                  /* temel durum */
    if (s[left] != s[right])
        return 0;
    return isPalindrome(s, left + 1, right - 1);   /* içteki kısım */
}

/* Çağrı: isPalindrome(s, 0, strlen(s) - 1) */
```

Soru 8 (derste sözlü, 02:12:50)

Bağlı listenin önbellekten yararlanamamasının temel sebebi nedir?

Çözüm (derste verildi, düzeltildi): Hoca, bağlı listenin heap bölgesini kullandığını ve öyle tasarlandığı için önbellekte kullanılmadığını söyledi. Doğrusu, heap verisi de önbelleğe alınır; sorun düğümlerin bellekte dağınık olması ve ardışık erişimde önbellek ıskalamasının sık olmasıdır (bkz. 11.8'deki ders sonrası not).

Soru 9 (derste sözlü, 02:14:00)

Pointer'ların varsayılan değeri `NULL` mıdır?

Çözüm (derste verildi): Hayır, varsayılan değer yoktur. `calloc` ayırdığı yeri sıfırlar, `malloc` sıfırlamaz; orada önceden ne varsa o kalır. Değişkenler ve pointer'lar her zaman açıkça başlatılmalıdır.

Dersin Toparlaması

Bu derste özyinelemenin mantığı faktöriyel ve dizi toplamı örnekleriyle kuruldu: fonksiyon aynı işi daha küçük bir problem için kendini çağırarak yapar ve mutlaka bir temel durumda durur. Özyinelemeli fonksiyon yazmanın üç adımı (temel durum, özyinelemeli durum, sonlanmanın garanti edilmesi), doğrudan ve dolaylı özyineleme, her çağrının stack'te yeni bir kopya oluşturmasından gelen yer ve zaman maliyeti ele alındı. Fibonacci örneği, birbirine bağlı alt problemlerde aynı değerlerin defalarca hesaplanmasıyla özyinelemenin kötü kullanımını gösterdi. İkili arama hem dizide olan hem olmayan eleman için izlendi ve özyinelemeli yazıldı; Hanoi Kuleleri öğrencilere alıştırma olarak bırakıldı. Ana mesaj, özyinelemenin yalnız gerçekten gerektiğinde (quicksort, merge sort, ağaç ve graf algoritmaları gibi) kullanılmasıdır.

Dersin son kısmında bağlı listelere giriş yapıldı: statik dizi ve dinamik bellek ayırmanın yetmediği, eleman sayısının belli olmadığı ve sık ekleme-silme yapılan durumlar için düğüm yapısı, head, tek yönlü bağlı liste, temel işlemler, avantajlar ve dezavantajlar anlatıldı. Hoca, ekleme işleminin nasıl yapıldığını ve bağlı listenin kullanımını sonraki derste göstereceğini; ağaç ve graf konularında bu yapının kullanılacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.